Portable, Seekable Random Streams for Parallel CPU Simulation

ANONYMOUS AUTHOR(S)

A long simulation that checkpoints its random generator and resumes on a different machine is relying on a property that `std::mt19937` does not have. The C++ standard fixes the text format of an engine's stream operators only partially, and the three major standard libraries disagree on both the numeric base of the digits and the order of the 624 state words. A checkpoint written under libstdc++ fails to load under the MSVC standard library, and loads without any diagnostic, at the wrong position, under libc++. This paper describes vphilox, a header-only C++20 library that removes the failure by changing what is stored. The generator is Philox4x32-10, a counter-based construction whose entire state is a key and a position, so a checkpoint is six 32-bit integers and a word offset, not a snapshot of engine internals. The same property gives constant-time seek and gives each worker an independent substream with no coordination, so a task draws the same numbers whatever the thread count and schedule. The cost of those properties is the question the design has to answer, because the earlier attempt to adopt Philox in a production machine-learning library was rejected on the grounds that scalar Philox runs at one tenth the speed of the Mersenne Twister. We report that the tenfold penalty did not reproduce on any of five processors measured, and that interleaving independent counters across SIMD lanes brings the generator to between 0.22× and 0.75× the cost per byte of `std::mt19937` on the bulk path. Aggregate cost per byte is flat across thread counts, and the first parallel limit we find is hyperthread co-location, not the generator.

CCS Concepts: • **Mathematics of computing** → **Mathematical software**; *Probabilistic algorithms*; Mathematical software performance; • **Software and its engineering** → **Software libraries and repositories**; • **Computer systems organization** → *Single instruction, multiple data*; • **Computing methodologies** → Parallel algorithms.

Additional Key Words and Phrases: Counter-based random number generation, Philox, portable serialized state, constant-time seek, SIMD vectorization, reproducible parallel streams, thread scaling, C++20, statistical testing

ACM Reference Format:
Anonymous Author(s). 2026. Portable, Seekable Random Streams for Parallel CPU Simulation. *ACM Trans. Math. Softw.* 0, 0, Article 0 (October 2026), 19 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Consider a Monte Carlo simulation that runs for a week on a cluster, writes a checkpoint every hour, and is restarted after a node failure onto whatever hardware the scheduler has free. For the restart to be scientifically meaningful, the random number generator must resume at exactly the position where it stopped. The replacement node need not resemble the one that died: it may run a different C++ standard library, carry a different instruction set, and be handed a different number of worker threads. A generator fit for this setting has to be indifferent to all three. In C++ the standard mechanism for resuming a generator is the stream insertion and extraction operators on the engine, and for `std::mt19937` that mechanism does not survive even the first of them.

Two independent defects combine. The first concerns the numeric base. The standard specifies that the insertion operator sets the decimal flag on the stream, and libstdc++ implements this by overriding a caller's `std::hex`; the Microsoft standard library does not, so a caller who has set hexadecimal formatting obtains hexadecimal digits there and decimal digits under libstdc++. Each library then reads the other library's digits in the wrong base. The second concerns the layout. libstdc++ writes the 624 state words in raw internal order followed by a position index; libc++ and

2026. ACM 1557-7295/2026/10-ART0
https://doi.org/10.1145/nnnnnnn.nnnnnnn

the Microsoft standard library write the same 624 words rotated into canonical order and write no position at all. The word counts agree, which is what makes the failure dangerous: a reader that accepts 624 integers has no way to detect that it has been handed a differently ordered array, so the restore succeeds and the simulation continues from a state that is not the one that was saved.

Neither defect is hypothetical. A model trained and serialized on Linux under XGBoost 3.3.0 was reported as failing to deserialize on Windows under the same library version, with the base mismatch surfacing as a hard error [2]; the report rules out a version mismatch, a damaged file, and a locale difference in turn. That failure is the recoverable case. The layout defect is the one that does not announce itself.

The failure is a consequence of serializing an implementation instead of a position. A Mersenne Twister state is a large array plus an index into it, and the array has no canonical form. A counter-based generator has no such array. Following Salmon et al. [9], the nth output block of Philox is a stateless bijection applied to the integer n under a key, so the complete state of a stream is the key and the value of n. Writing that down is unambiguous, and reading it back cannot silently land somewhere else.

The same structure answers the remaining two axes without additional machinery. Because the bijection is fixed, any implementation that computes it correctly emits the same words, so a checkpoint written on a node running the AVX-512 kernel resumes on a node running the NEON one: how a host vectorizes the round function is an implementation detail the stream cannot observe. And because two disjoint counter ranges are two independent substreams, a work-partitioned program consumes the same random numbers in the same positions regardless of how many threads it runs on or in what order the scheduler happens to run them. Seeking comes free with it, since advancing a stream by N blocks is an addition on a 128-bit integer, so `discard` runs in $O(1)$, not in time proportional to N.

These properties are not new; they are the reason counter-based generators were proposed. What has kept them out of some CPU codebases is a performance objection. A proposal to make the serialized generator state of XGBoost, a widely used gradient boosting library, portable across platforms by replacing `std::mt19937` with a Philox engine was declined on throughput grounds: "[t]he performance here is 1/10 of mt. This is due to no specialisation for wide multiplies" [6].[1] That objection is the thing this work has to remove, and it sets the shape of the evaluation: the target is not to be the fastest generator available, but to make the portability properties cost nothing.

This paper makes four contributions.

(1) A generator whose output is invariant across the three axes above: thread count, thread scheduling, and the instruction set of the host (Section 7.2). A partitioned run on four threads and on sixty-four consume identical numbers in identical positions, and the AVX-512 kernel's stream is bit-identical to the scalar reference's. A generator with shared state offers this only by locking, or not at all.

(2) A parallel characterisation of what that costs (Section 7). Aggregate cost per byte is flat with thread count to one thread per physical core, and the first limit on a 16-core host is hyperthread co-location at 35% per worker, not memory bandwidth. Instruction supply is excluded by measurement rather than argument: co-location costs 59% more per byte while L1 instruction-cache misses *fall* by 36%.

(3) A portable serialized state (Section 3) that records a position and no engine internals, refuses malformed input instead of interpreting it, and does not consult the locale. A change to the

[1] The authors are not independent of the exchange quoted here. The disclosure is on the title page supplied to the editor. The measurements reported here were made after that exchange, and Section 9 states the cases in which they do not support the design.

engine's internal buffer size, from 8 blocks to 16, left previously written states readable, which is the property being claimed.

(4) Counter-interleaved SIMD kernels for AVX2, AVX-512 and ARM NEON (Section 4), and a measurement of the objection that motivated them (Section 6). On five processors, unspecialised scalar Philox costs between 1.10× and 1.65× per byte what `std::mt19937` costs, not 10×; with the multiplies specialised the bulk path costs between 0.22× and 0.75×.

We also report two results that go against the library, in Section 9: xoshiro256++ is faster on four of the five machines measured, and on ARM the buffered engine interface remains slower than `std::mt19937` even though the NEON kernel beneath it is faster.

2 Background

2.1 Counter-based generation

A conventional generator holds internal state s and computes $(s_{i+1}, x_i) = f(s_i)$, so obtaining x_n requires n applications of f. A counter-based generator instead defines

$$x_n = b(\mathbf{c}_0 + n, \mathbf{k}), \tag{1}$$

where b is a keyed bijection, $\mathbf{c}_0$ is a starting counter and $\mathbf{k}$ is a key. Nothing carries between successive outputs. The three properties used throughout this paper follow directly from Equation (1): the state is $(\mathbf{k}, n)$ and nothing else, evaluating x_n for arbitrary n costs the same as evaluating x_0, and disjoint ranges of n are independent.

2.2 Philox4x32-10

Philox4x32 takes a 128-bit counter, held as four 32-bit words $\mathbf{c} = (c_0, c_1, c_2, c_3)$, and a 64-bit key $\mathbf{k} = (k_0, k_1)$. One round performs two $32 \times 32 \longrightarrow 64$ multiplications, splits each product into its high and low halves, and permutes the words:

$$(h_0, l_0) = \mathrm{mulhilo}(M_0,\ c_0), \tag{2}$$

$$(h_1, l_1) = \mathrm{mulhilo}(M_1,\ c_2), \tag{3}$$

$$\mathbf{c}' = \big(h_1 \oplus c_1 \oplus k_0,\ l_1,\ h_0 \oplus c_3 \oplus k_1,\ l_0\big), \tag{4}$$

with M_0 = `0xD2511F53` and M_1 = `0xCD9E8D57`. Between rounds the key advances by a Weyl sequence, k_0 += `0x9E3779B9` and k_1 += `0xBB67AE85`, which breaks the symmetry that would otherwise make every round identical. Ten rounds are applied; seven already pass BigCrush [9], so the standard variant carries a margin.

The whole construction is integer multiply, exclusive or, addition and word permutation. It contains no floating-point instruction, a fact that becomes relevant in Section 6.4.

2.3 The one invariant

Every design decision below is subordinate to a single rule: for a given key and counter, vphilox produces exactly the bytes that the Random123 reference implementation produces, across compilers, architectures and library versions. The published test vectors of Salmon et al. [9] are checked directly, and Section 7.2 describes the digest tests that cover what those vectors cannot. A change that moves the digest is a different generator, not a new version of this one.

3 Portable serialized state

3.1 What is written

The serialized form records a position, defined as the key, the counter of the block containing the next output, and the index of the word within that block:

```
vphilox1 <k0> <k1> <c0> <c1> <c2> <c3> <word>
```

Seven decimal integers and a version tag. Every field is an unsigned 32-bit value except the final word index, which lies in $[0, 4)$. The whole position occupies 28 bytes in memory and at most 76 characters on disk, against `std::mt19937`'s 5000 bytes under libstdc++ on x86-64 – `uint_fast32_t` is 64 bits there while the state words are 32-bit values – and roughly 6.7 kB of decimal text serialized. Recording a position is two orders of magnitude smaller, and unlike the array its size does not depend on which standard library wrote it.

The distinction between this and the engine's internal variables is the point of the design. The engine generates into a buffer of `refill_blocks` blocks at a time and hands out words from it, so its internal counter runs ahead of the caller-visible position by however much remains buffered. Serializing the internal counter and buffer cursor would embed `refill_blocks` in the file format. That constant changed from 8 to 16 when the AVX-512 kernel landed, because a refill must not be split across a kernel invocation. States written before the change remain readable after it, which is the property a checkpoint format has to have and the one `std::mt19937` lacks.

Recovering the caller-visible position requires the inverse of the counter advance, since the internal counter must be retreated by the number of blocks still buffered. This is the reason the 128-bit counter arithmetic in the library implements subtraction as well as addition.

3.2 Two rules the writer follows

The locale is never consulted. Integer extraction through `std::istream` routes through the `num_get` facet of the global locale, which may be imbued with digit grouping. The library formats and parses digits by hand; the only stream operations performed are on characters. A program that has imbued a grouping locale for its user interface therefore cannot corrupt its own checkpoints.

Unrecognised input is refused, not interpreted. Parsing fails on a wrong version tag, a wrong field count, a value exceeding $2^{32} - 1$, a word index outside a block, or trailing characters. The contrast with the `std::mt19937` case is exact: there, a valid-looking array of 624 integers in the wrong order is accepted, and the generator resumes at a position nobody chose. Here, a state that cannot be read produces an error the caller can observe.

3.3 Constant-time seek

Because block n is $b(\mathbf{c}_0 + n, \mathbf{k})$, advancing the stream by N blocks is the addition $\mathbf{c}$ += N on a 128-bit little-endian integer. The implementation propagates carries across four 32-bit words, so `discard(N)` costs the same for $N = 1$ and for $N = 2^{40}$. The corresponding operation on a Mersenne Twister requires either N state transitions or a jump polynomial computed offline.

Remark 1. The engine exposes both `counter()` and `state()`, and they are not interchangeable. Constructing an engine from `key()` and `counter()` skips whatever the buffer still holds, because `counter()` is the counter of the *next refill*. Only `state()` and `set_state()` form an exact round trip, and a test asserts that the other route skips, so the difference is recorded as behaviour and not left as a trap.

4 Vectorization by counter interleaving

4.1 The kernel contract

Every backend implements one function. Given a base counter $\mathbf{c}_0$, a key $\mathbf{k}$, an output pointer `out` and a block count B, it writes $4B$ unsigned 32-bit words, block i being $\mathrm{philox4x32}_{10}(\mathbf{c}_0 + i, \mathbf{k})$ at `out`$[4i \ldots 4i + 3]$.

Table 1. Interleaving width per backend. On x86 the width is one counter per 32-bit lane. NEON carries four lanes per register but two independent groups per iteration, for the reason given in Section 4.3.

Backend	`preferred_blocks`	Layout
scalar	1	reference implementation, fully `constexpr`
AVX2	8	8 counters per `__m256i`
AVX-512	16	16 counters per `__m512i`
NEON	8	4 counters per `uint32x4_t`, 2 groups interleaved

Two requirements attach to it. Output must not depend on how the caller divides a request, so generating B blocks in one call and in several calls of summed length B must produce identical bytes. And the kernel handles its own tail, since `out` carries no alignment or length guarantee. The first requirement is what makes cross-backend parity testing meaningful, and it is what allows the engine to choose any refill size.

Each backend also advertises `preferred_blocks`, the block count at which it runs without a tail. Table 1 lists the four.

4.2 Why the multiplies are the difficulty

Equations (2) and (3) require the full 64-bit product of two 32-bit values, which scalar code obtains from a single widening multiply. Vector instruction sets offer no 32-lane widening multiply that keeps both halves in place: on x86, `vpmuludq` multiplies the even 32-bit lanes and writes 64-bit results, so producing W independent products from $2W$ packed values takes two multiplies, one on the even lanes and one on the odd after a shift, then a repack.

A naive vectorization at four counters per 256-bit register therefore leaves half the lanes idle during the exclusive-or, key broadcast and permutation steps of Equation (4) while still paying for the lane management. Filling all eight 32-bit lanes amortises that management across twice as many counters: measured directly on an Intel Core i5-11300H, the eight-counter layout is 21.1% faster, and it is the one used. AVX-512 applies the same reasoning at 16 counters.

4.3 NEON is latency-bound, not width-bound

The first NEON kernel used one group of four counters per iteration and reached 2.2035 cycles per byte on a Cortex-A76, 1.46× the scalar kernel but only 0.89× `std::mt19937`. Width was not the constraint: the ten rounds form a serial dependency chain, and one four-lane group issues its multiplies faster than their latency retires them, so the pipeline drains between rounds. Carrying two independent groups per iteration and alternating their rounds fills that latency with the other group's work, giving 1.4652 cycles per byte, a 1.504× speedup that places vphilox ahead of both `std::mt19937` (1.33×) and PCG64 (1.11×) there. This is why `preferred_blocks` for NEON is 8 and not 4: it reflects the work per iteration, not the register width.

4.4 Dispatch

A backend is selected only if it was compiled in, is marked implemented, and is supported by the running processor. Feature detection uses a direct `CPUID` and `XGETBV` probe written once per compiler, not a compiler builtin, so the Linux and macOS builds exercise the same detection path that the Microsoft build depends on. Resolution happens once per round count on first use.

Instruction set flags are never applied to the whole build. Kernels carry per-function target attributes, so one binary starts and runs correctly on a processor lacking the instructions and falls

back at run time. This matters for the distribution model: a header-only library that required an `-mavx512f` build would push the portability problem from the checkpoint file to the binary.

4.5 Floating-point conversion

The obvious conversion from a random word to a value in $[0, 1)$ divides by 2^{32}, which costs a conversion instruction and a division. The library instead constructs the result bit pattern directly. For single precision, setting the sign bit to zero and the exponent field to 127 and placing 23 random bits in the mantissa yields a value uniform on $[1, 2)$; subtracting one maps it to $[0, 1)$:

$$\text{to_float01}(u) = \text{bitcast}\big(\texttt{0x3F800000} \mid (u \gg 9)\big) - 1.0f. \tag{5}$$

The double precision form is identical with exponent 1023 and 52 mantissa bits, differing only in where the bits come from: the generator emits 32-bit words, so a double is assembled from two of them concatenated into a 64-bit value, of which the top 52 bits are injected and the low 12 discarded. On the vector path this is a bitwise or followed by a subtract.

The cost is resolution, not correctness. The injection fills the 23 stored mantissa bits, so outputs are uniform on a grid of spacing 2^{-23} and 9 of the 32 random bits are discarded. A float carries 24 significand bits, but the 24th is the implicit leading one, which the fixed exponent pins and which therefore carries no entropy. Callers who need all 32 bits should take the integers.

How much a vector form of Equation (5) is worth depends on where the conversion loop is bound. Measured against working set on Sapphire Rapids, an AVX-512 conversion loop is 1.89× the scalar one at the 256-word tile the library converts in and 3.19× at 32 MiB, converging with it at 512 KiB where the loop is bandwidth-bound and width buys nothing. The width therefore pays at the tile size the library actually converts in, which is the case for putting the conversion on the vector path.

Both steps of Equation (5) are exact. The subtraction is exact by Sterbenz's lemma, since the operands lie within a factor of two of each other, and adding the value back is exact because $k \cdot 2^{-23}$ needs only 23 significant bits. Every injected bit therefore survives a round trip, and this is verified exhaustively over all 2^{23} representable outputs, not by sampling.

5 Experimental method

5.1 What is measured, and in what unit

Results are reported first in cycles per byte and second in bytes per second, cycles being read from the timestamp counter on x86 and through `perf_event_open` on Linux ARM. The unit removes the clock from comparisons between generators on one machine, which is what most of the questions here are about.

It does not remove the clock from comparisons *between* machines. The x86 timestamp counter increments at a fixed reference frequency, not the core frequency, and the ARM counter is a different counter entirely. Cycles per byte is therefore compared only within a single host. Any figure spanning several machines plots a ratio formed inside each machine, which is dimensionless and does transfer. Absolute values appear only in tables that carry the machine name beside them.

5.2 The measurement protocol

Runs are driven by a script and never by hand, for reasons each learned by getting the measurement wrong first. The build is unconditional, because a stale build directory will run a months-old binary and print a plausible table, and the script asserts that the expected rows exist before accepting a run. The configuration is the same on every host: a release build at `-O3` with assertions disabled, no link-time optimisation, and no instruction set flag on the translation unit. Kernels take their instruction sets from per-function target attributes instead, so the backends compared in a table

are compiled into one binary and chosen at run time, and a comparison between them cannot be confounded by how they were built. The frequency governor is pinned and restored on exit, and the resolved backend, compiler, processor model, commit hash and verbatim command are recorded beside the numbers. The backend cannot be recovered after the fact: three early archived runs were taken on AVX-512 hardware while that kernel was still a stub, so the processor model does not determine what executed.

A run is rejected if the cycle counter did not report or if any row exceeds a coefficient of variation of 1% in cycles per byte. Four archived runs do not meet the bar; they carry a quality label, are excluded from every figure, and are kept because they built the harness. Exactly one row is retained above the bar: the co-located arm of Table 6, at 2.22%. The bar exists to keep a noisy number out of a comparison of absolute costs, and what that run contributes is the sign and order of a change in a ratio rather than a level. Cloud instances are used for throughput only when the instance has dedicated virtual CPUs and the run still clears the bar, and the write-up says so.

Remark 2 (Virtual machines cannot place threads). A guest publishes a correct-looking sibling map and accepts every affinity mask, while the hypervisor schedules virtual CPUs onto host cores on its own. The mask then fixes virtual identifiers and nothing physical. Under WSL2 we measured two workers on one core's siblings as *faster* than two on separate cores (0.63 against 0.63 to 0.77 cycles per byte, and stable against a 20% spread), which is impossible for an execution-port-bound kernel under real co-location. The instruction-supply study in Section 7.5 therefore runs on bare metal, and gates itself on a measured co-location penalty before it will report anything. The larger placement comparison in Section 7.4 does not: it runs on a cloud instance. What makes a mask meaningful there is that the provider allocates each virtual CPU as a dedicated hardware thread rather than time-slicing physical cores between tenants, and exposes the host's sibling and NUMA topology to the guest, so a mask naming two siblings names two real ones. A guest cannot confirm from the inside that it holds the whole machine, and a neighbouring instance's activity stays invisible, so this is weaker than bare metal and is treated as such: the mechanism in Section 7.5 is not taken from that host.

5.3 Machines

Table 2 lists the six hosts. The two Cascade Lake entries are the same processor, family 6 model 85 stepping 7, in a four-vCPU slice of a shared socket and on an instance reporting two whole sockets; they are reported separately because cycles per byte does not transfer between hosts and these two behave as two hosts. Five carry the throughput results in Section 6. The sixth, a Tiger Lake laptop, carries only the studies that need physical thread placement, for the reason stated in the remark above: it is the one multi-core host on record that is bare metal and exposes a working performance monitoring unit at once. Its own throughput matrix is archived as harness validation and quoted nowhere, because a four-core machine running a full desktop cannot hold the variation bar across a whole sweep.

Turbo is disabled on the Tiger Lake host so that reference cycles and core cycles coincide: the timestamp counter ticks at that part's base clock, so with boost off cycles per byte is a true core-cycle count and a moving turbo ceiling cannot land in the column. The cost is that its absolute numbers are approximately 1.4× the same machine's boosted numbers and are comparable only within that host, which is all either study asks of them.

5.4 Reproducibility of the figures

Every table and figure here is generated from archived JSON by one standard-library-only script, which continuous integration re-runs in checking mode, so a run archived without regenerating

Table 2. Measurement hosts. Two are bare metal and four are cloud instances with dedicated virtual CPUs. The 32-vCPU Cascade Lake entry reports a two-socket, sixteen-core topology to the guest, which is what makes a placement comparison admissible on it; see the remark in Section 5.

Label	Processor	MHz	Backend	Used for
Pi 5 / NEON	Raspberry Pi 5, 4-core Cortex-A76, bare metal	2400	neon	throughput, scaling
Sapphire Rapids	Intel Xeon Platinum 8481C, dedicated vCPU	2700	avx512	throughput
Skylake-SP	Intel Xeon, dedicated vCPU	2000	avx512	throughput
Cascade Lake 4v	Intel Xeon, 4 dedicated vCPU	2800	avx512	throughput
Cascade Lake 32v	Intel Xeon, cloud, 32 vCPU, guest reports 2 sockets, 16 cores	2800	avx512	throughput, placement
Tiger Lake	Intel Core i5-11300H, 4 cores × 2 threads, bare metal, no turbo	3100	avx512	placement mechanism

Table 3. Cost per byte relative to `std::mt19937` on the same machine. Bold marks the lowest cost achieved by vphilox in each column.

	Pi 5 NEON	Sapphire Rapids	Skylake-SP	Cascade Lake 4v	Cascade Lake 32v
xoshiro256++	0.28	0.30	0.24	0.17	0.29
PCG64	0.83	0.59	0.33	0.32	0.53
`std::mt19937`	1.00	1.00	1.00	1.00	1.00
Philox, unspecialised	1.65	1.55	1.15	1.10	1.62
vphilox, `operator()`	1.15	0.79	0.51	0.58	0.68
vphilox, `generate_n`	**0.75**	**0.38**	**0.22**	**0.24**	**0.33**

the derived files fails the build. The script writes its own vector graphics as fixed-precision text, which is what makes them byte-reproducible and therefore checkable.

6 Single-threaded throughput

6.1 The comparison matrix

Table 3 gives cost per byte for each generator divided by `std::mt19937` on the same machine. Lower is faster.

6.2 The wide-multiply objection

The row that answers the objection in Section 1 is the fourth. Scalar Philox without specialised wide multiplies costs between 1.10× and 1.65× per byte what `std::mt19937` costs. The reported figure was 10×. It did not reproduce on any of the five machines, and on the four-vCPU Cascade Lake it is within 10% of parity.

We do not claim the original measurement was wrong, only that it does not describe the comparison as made here; a plausible explanation is that it went through a heavier abstraction layer than the generator itself, which is a property of the integration rather than of Philox.

With the multiplies specialised, the bulk path costs between 0.22× and 0.75×. The objection is removed twice over: once by measuring the unspecialised case honestly, and once by specialising it.

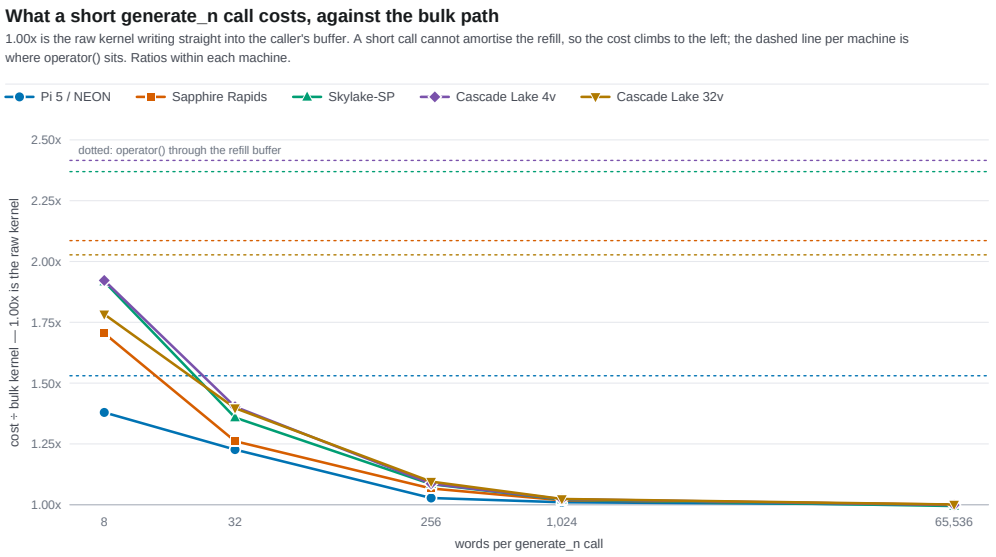


Fig. 1. Cost of a bulk call relative to the raw kernel on the same machine, against call size in words. The dashed line per machine is the per-call interface through the refill buffer.

6.3 The refill buffer is the remaining cost

The gap between the two vphilox rows in Table 3 is the refill buffer. Satisfying `std::uniform_random_bit_generator` requires a per-call interface, so the engine generates into an aligned 64-word buffer and hands out words; the bulk entry points run the kernel straight into the caller's buffer and skip the copy.

That gap widens as the kernel gets faster: 1.53× on the Pi, 2.03× on Cascade Lake 32v, 2.09× on Sapphire Rapids, 2.37× on Skylake-SP and 2.42× on Cascade Lake 4v. A second pass over every byte has a fixed cost per byte, so the faster the kernel becomes, the larger the fraction of total time that drain represents. Expressed the other way, the buffer costs approximately 109% of bulk throughput on AVX-512, 42% on AVX2 and 10% on x86 scalar.

Figure 1 shows the practical form of this. A bulk call of eight words costs 1.4× to 1.9× the raw kernel; by 256 words the remainder is a few percent, and by 65536 words it has gone. The two fastest kernels pay the most for a call too small to amortise the setup, which is the same effect seen from the other direction.

6.4 AVX-512 and the frequency licence

Intel processors reduce core frequency when executing 512-bit instructions, and on Skylake-SP that reduction was large enough to make AVX-512 a net loss for some workloads. The concern applies here, since dispatch prefers the widest available kernel. It does not appear: Table 4 gives bulk cost per byte with the backend pinned, same binary in each row.

What the table establishes is an absence: on every part measured here the 512-bit kernel wins, and on the one part whose cores are not shared a probe recording core frequency alongside the benchmark shows the clock flat across the range in question. The explanation consistent with that, and with Intel's documented licence behaviour, is that the licences are tiered by the kind of 512-bit instruction executing rather than by width alone, with heavy floating-point and fused multiply-add

Table 4. Bulk kernel cost per byte by pinned backend, cycles per byte, same binary in every row. Width conversion is the AVX-512 speedup as a fraction of the doubled lane width, so 100% would be perfect scaling with register width.

Machine	scalar	AVX2	AVX-512	vs AVX2	width conv.
Skylake-SP	2.1753	0.7698	**0.4202**	1.83×	92%
Sapphire Rapids	1.9311	0.8462	**0.4712**	1.80×	90%
Cascade Lake 32v	2.2883	0.8740	**0.4747**	1.84×	92%
Cascade Lake 4v	2.4142	0.8537	**0.5466**	1.56×	78%

work dropping the core furthest and light integer, logical and shuffle work staying at or near the AVX2 licence. As noted in Section 2, the Philox inner loop is `vpmuludq`, `vpaddd`, `vpxord`, `vpsrlq` and lane shuffles, with no floating-point instruction anywhere in it.

That is an attribution rather than a measurement. Isolating instruction type would need the same kernel rebuilt from heavy 512-bit instructions on the same parts, which was not run, so the instruction mix remains one difference among several between these hosts and the ones where the penalty was reported. Read as a hypothesis it predicts that counter-based generators built on integer multiplication are good candidates for 512-bit vectorization, and nothing measured here contradicts it.

One row of Table 4 looks at first like the penalty and is not. A four-vCPU slice of a Cascade Lake socket converts 78% of the doubled width, against 90% to 92% everywhere else, but that is a property of the instance and not of the part: the same stepping on the sixteen-core instance converts 92%, and on the shared slice the *scalar* kernel is slower as well, at 2.4142 against 2.2883 cycles per byte for code containing no 512-bit instruction at all. No frequency licence can produce a scalar slowdown. The consistent explanation is that other tenants move the package turbo budget underneath a measurement the timestamp counter reports in reference cycles, which is the risk described in Section 5.

That reading is not confirmed, and the 4-vCPU figure is reported, not discarded. What the hosts jointly support is narrower and sufficient: AVX-512 beats AVX2 on every part measured, by 1.56× at worst and 1.84× at best, so the dispatch preference stands, and on a part whose cores are not shared a probe recording core frequency alongside the benchmark shows it flat across the range in question.

7 Parallel behaviour

7.1 Partitioning the counter space

Independence between workers is arranged by construction rather than by synchronisation. One key is fixed for the whole run and worker t of P is given a contiguous run of B blocks, so its ith block is

$$\mathbf{c}(t, i) = \mathbf{c}_0 + tB + i, \qquad 0 \le t < P, \;\; 0 \le i < B. \tag{6}$$

The intervals $[tB, (t+1)B)$ are disjoint and together cover $[0, PB)$, so the workers consume a partition of one stream rather than P streams that are merely unlikely to collide. Because $b(\cdot, \mathbf{k})$ in Equation (1) is a bijection for a fixed key, distinct counters give distinct outputs, and the non-overlap follows from the partition with nothing assumed about the generator beyond that. Each worker owns its engine and reads the key, which is immutable for the run, so there is no lock and no atomic on the generator; the one hazard that remains is physical rather than algorithmic, and engines are given separate allocations so that two of them cannot share a cache line. The counter is 128-bit, so

partitioning does not become the scarce resource: 2^{32} workers still leave each of them 2^{96} blocks. The measurements in this section use Equation (6) directly, with B the number of blocks a worker fills and the offset formed by the same 128-bit addition that backs `discard`.

Equation (6) indexes by worker, which is the right mapping for a benchmark that fills P buffers and the wrong one for a simulation. A simulation's unit of work is a task, and the number of tasks is a property of the problem where P is a property of the machine; if the counter offset is derived from the worker index, changing P changes which draws a task receives even though the concatenated stream is unchanged. The mapping a caller wants is therefore from the task index j directly,

$$\mathbf{c}(j, i) = \mathbf{c}_0 + jS + i, \qquad 0 \le i < S, \tag{7}$$

with S a stride fixed at or above the largest number of blocks any task can consume. Worker t then holds no counter state of its own: it constructs an engine at $\mathbf{c}(j, 0)$ for whichever task it claims and discards it when that task ends. The assignment of draws to tasks is then a function of j alone, so it survives a change of P, a change of schedule, and a change in the order tasks complete. Equation (6) is the special case in which tasks are handed out in contiguous equal blocks and $S = B$.

Sizing S too small is the failure mode, since overlapping intervals reintroduce the correlation the partition exists to exclude. The counter is 128-bit, so the safe choice is generous: 2^{32} tasks at a stride of 2^{32} blocks still fits.

The alternative partition gives each worker a distinct key and leaves the counters equal. It is not used here, and the reason is the strength of the guarantee rather than the cost. Distinct keys select distinct bijections, and nothing in the construction prevents two of them from carrying their equal counters to overlapping output; disjointness would become an empirical expectation where Equation (6) makes it an identity. Keying by thread index also makes the stream a function of the thread count, which is the property the next subsection depends on not having.

7.2 Output does not depend on the configuration

The property that motivates the parallel work is not throughput, so it is stated before any timing, and it comes at two levels that are not the same claim. At the level of the stream, the concatenated output of a partitioned run is identical for every P and every scheduling, because worker t takes the counter range fixed by Equation (6) rather than the range the order of arrival would give it. At the level of the simulation, task j draws the same words whatever worker runs it and whatever else is running, which follows from Equation (7) and not from the stream property: a run that concatenates to the same bytes can still hand those bytes to different tasks, and a caller who partitions by worker index gets the first guarantee without the second. A generator with shared state offers the first only by serialising access behind a lock, and the second not at all.

The same holds across the hardware. The kernel contract of Section 4 requires every backend to emit what the scalar reference emits, independent of how a request is chunked, so the four backends are interchangeable at the level of the stream. A run that begins on a node dispatching to AVX-512 and resumes from a checkpoint on a node dispatching to NEON continues the same sequence. Two tests carry it, covering different halves. The first folds an 8191-block stream, a length chosen to straddle every backend's interleaving width, into a digest checked against a constant compiled into the test. The second pins a serialized state string produced on x86 and requires that restoring it and drawing 4096 words reproduces a fixed digest, which is the checkpoint half: the machine reading the state is not the machine that wrote it. Continuous integration runs both on Linux, Windows and macOS arm64, so an x86-written state is restored on an ARM host on every commit.

Those two run on one thread, so a third covers the thread axis. It fixes a workload of 97 tasks of unequal length – with equal lengths a dynamic schedule hands out what a static one hands out and

Table 5. Raspberry Pi 5, four cores, 256 KiB per worker, medians. Cost per byte is Equation (8); speedup and efficiency are relative to one worker.

Workers	Cyc./byte	GB/s	Speedup	Efficiency
1	3.1953	0.733	1.00×	100.0%
2	3.1948	1.464	2.00×	99.9%
4	3.1950	2.896	3.95×	98.8%
8	3.1954	2.818	3.85×	48.1%
32	3.1954	2.776	3.79×	11.8%

the schedule axis tests nothing – maps them by Equation (7), and compares every task's words against a serial reference for worker counts 1, 2, 3, 5, 8 and 13 under two schedules: contiguous blocks, and tasks claimed one at a time from an atomic counter, so which worker runs which task is decided by the operating system and differs between runs of the same binary. All twelve configurations reproduce the serial result word for word. The same test splits a fixed 240 blocks across nine worker counts and checks the concatenation is invariant and equal to what one engine produces in a single call.

Together with the serialized state of Section 3, this is what lets a checkpoint move between unlike nodes. The remainder of this section measures what the property costs.

7.3 Cost per byte does not move with thread count

Three quantities move differently here and only one of them is plotted. Aggregate cost per byte is

$$\bar{c} = \frac{\sum_{t<P} C_t}{\sum_{t<P} N_t}, \tag{8}$$

where C_t is the cycles worker t spent inside its own fill and N_t the bytes it produced. It sums per-thread counters rather than multiplying elapsed wall time by a clock, which is what makes it indifferent to whether the workers ran concurrently. Wall-clock throughput is $\sum_t N_t$ over elapsed time, and parallel efficiency is speedup divided by P. Counter-based generation partitions with no shared state, so $\bar{c}$ should not move with the worker count, and on the Raspberry Pi 5 it does not: across 1 to 32 threads and two working sets it spans 0.063%, from 3.1934 to 3.1954 cycles per byte. Table 5 puts the other two columns beside it.

Throughput rises to four threads and then stops, because the machine has four cores, and efficiency collapses past that point as it must once P exceeds the core count. The flat curve and the plateau are not in tension: they are not measurements of the same thing. Equation (8) counts the cycles spent producing a byte, and oversubscription does not change that count, only how many cycles per second of wall time each worker is granted. Thirty-two threads on four cores produce a byte for the same 3.1954 cycles one thread does and wait eight times as long for them. A flat $\bar{c}$ is evidence that the generator adds no coordination cost as workers are added, which is the claim; it is not evidence of linear speedup, which needs cores as well. Figure 2 draws $\bar{c}$ for both working sets.

Four cores are too few to reach a knee, which is why the placement study exists.

7.4 The first limit is hyperthread co-location

On a two-socket, sixteen-core Cascade Lake host the curve does bend, and Figure 3 separates the cause from the correlate. With one worker per physical core, aggregate cost per byte is flat from 1 to 16 cores, moving from 0.4750 to 0.4833 cycles per byte, a rise of 1.7%. Placing two workers on one core's hyperthread siblings costs 0.6536 cycles per byte, approximately 35% per worker.



Fig. 2. Aggregate cost per byte, Equation (8), against thread count on the Raspberry Pi 5. Four series are drawn and two are visible, because each working set lands on the other. Wall-clock throughput is not on this axis and does not stay flat; Table 5 carries it.

Table 6. Four workers under two placements, Tiger Lake, turbo disabled. Instruction supply improves where the penalty appears.

Placement	Cyc./byte	i$ MPKI	Instr./byte
One per physical core	0.5160	0.2108	0.958554
Packed onto siblings	0.8204	0.1356	0.958554
Change	+59.0%	−35.7%	±0.000000

The flat arm also shows the mask is honoured, without appealing to the size of the penalty it measures. Workers added under a mask that named siblings only by accident would start sharing cores well before the sixteenth and the curve would bend where they did. It stays flat to sixteen and bends only when the mask packs siblings deliberately, which a guest whose affinity had no physical effect could not produce.

The mechanism is that the kernel is execution-port-bound. Two hardware threads sharing one core's multiply ports do not have two cores' worth of issue capacity, and Philox spends its time in exactly the ports they contend for. Memory enters as a second and later limit, and it tracks footprint per socket against the 33 MiB last-level cache instead of tracking thread count. Frequency is flat across the whole range.

7.5 Instruction supply is not part of the penalty

Sibling threads share one first-level instruction cache as well as the execution ports, which makes instruction supply a competing explanation for the same penalty. Table 6 separates them. Four workers run the identical job twice, changing only placement, with retired-instruction and level-one instruction-cache read-miss counters attached to each worker.

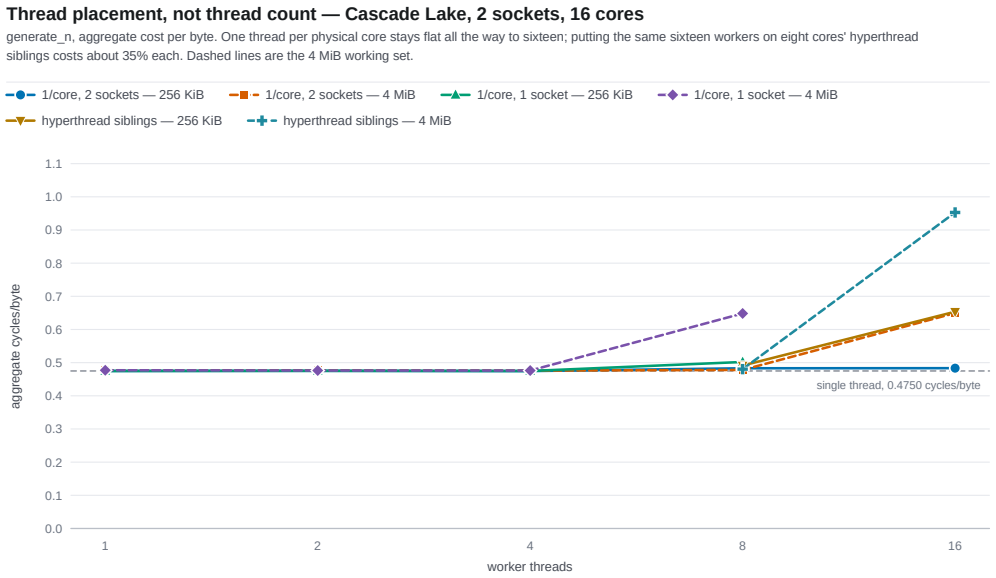


Fig. 3. Aggregate cost per byte against thread placement, Cascade Lake, two sockets, sixteen physical cores. Dashed lines are the 4 MiB working set.

Co-location costs 59% more cycles per byte while instruction-cache misses per thousand instructions *fall* by 36%. What that supports is a negative. An explanation in which the penalty comes from instruction supply requires misses to rise where the penalty appears; they move the other way, so that explanation does not fit these data. It does not by itself establish port contention, which remains an inference from the instruction mix rather than a measurement: the inner loop is `vpmuludq`, `vpaddd`, `vpxord`, `vpsrlq` and lane shuffles, and two hardware threads on one core issue those through the same ports. Separating the two directly would need per-port issue counters under the same two placements, which this part's counter set does not expose. The direction of the miss rate is at least unsurprising: both siblings execute the same kernel over disjoint counter ranges, so the second thread finds the code already resident, and a workload whose threads ran different code would not get that for free.

The third column is the check that the counters were attached to the correct threads. Instructions per byte is a property of the kernel and cannot move with placement, and it reads identically to six decimal places in both arms at zero variation across fifteen repetitions. This matters because the obvious instrumentation is wrong here: a benchmark framework that starts and stops counters on the thread running the benchmark loop would have counted the dispatcher, which blocks on a condition variable while the workers execute every instruction. Misses per thousand instructions is also a per-instruction ratio and therefore clock-independent, and it agrees to about 1% between the turbo-enabled and turbo-disabled runs, across a 42% change in clock.

The second guard is a gate, with a cost as well as a purpose. The study refuses to run where two workers on one core's siblings do not cost at least 15% more than two on separate cores, because the absence of that penalty is what an affinity mask that moved nothing looks like, and the two arms would then be one placement measured twice. It is a control rather than a result, and it admits only hosts on which co-location is expensive, so the 59% in Table 6 is not an unbiased estimate of

Table 7. Aggregate cost per byte by threading runtime, Tiger Lake, 256 KiB per worker. The × columns are normalised within each runtime.

Workers	`std::thread`	×	OpenMP	×
1	0.5084	1.000	0.5086	1.000
2	0.5165	1.016	0.5086	1.000
4	0.6205	1.220	0.5102	1.003
8	0.8076	1.588	0.9126	1.794

that penalty on an arbitrary machine and is not read as one. What the gate does not select on is the miss rate, which is what this subsection is about.

A threshold on its own is not sufficient, and a WSL2 guest shows why: the same two-worker comparison returned +23.7% on one invocation and −6.5% on the next, so a single probe can clear 15% on scatter alone. The gate therefore also requires the gap to exceed three times the worse arm's coefficient of variation, roughly a three-sigma separation from zero. The archived study clears that by a factor of nine, at 59.0% against a worst-arm variation of 2.22%.

A third host, taken after the gate gained that second condition, is what we would point a reader to rather than either of the first two. Eight workers on a second sixteen-core Cascade Lake instance cost 0.4782 cycles per byte one per physical core and 0.6873 packed onto four cores' siblings, a penalty of 43.7% through a gate that recorded 20.9% against a worst-arm variation of 0.79%. What it adds is the record rather than the number: its environment files carry the affinity mask each arm actually ran under, which the earlier runs do not, so the placement is documented rather than argued. Those masks also show the arms unmatched on socket spread, two cores and six against four and four, which at 2 MiB of working set against a 33 MiB last-level cache is not a load placement can change, but is recorded as a difference other than co-location. And the host contradicts something the first two suggested: there the co-located arm was the noisier, 2.22% against 0.67% in Table 6, where here it is the quieter, 0.25% against 0.42%. That asymmetry is not a dependable signature of contention and is not read as one. The instance exposed neither a governor nor turbo control, so its absolute cycles per byte are quoted nowhere; a ratio between two arms measured minutes apart does not need a pinned clock, and that ratio is all this paragraph uses.

Placement on that host is witnessed independently of the penalty too, if indirectly: had the mask failed to move the workers every column would have matched, as instructions per byte does, and the miss rate instead differs by 36%. That reads the miss rate as evidence about placement rather than about a mechanism, so it does not assume what it checks.

The co-located arm's variation, 2.22%, exceeds this paper's 1% bar and is retained under the exception in Section 5, because what is taken from it is the sign of a change in a ratio rather than a level: the median reproduces to 0.04% across two independent runs, and a 2% error bar does not reach a 36% effect.

7.6 The scaling result is the generator, not the harness

Every number above is measured through one persistent worker pool built on `std::thread`, and a benchmark cannot distinguish a property of the generator from a property of its own scaffolding. Running the identical job under a second, independent threading runtime does distinguish them. Table 7 repeats the sweep under OpenMP, with persistent teams on both sides so that neither column carries thread creation cost, and normalises each runtime to its own single-worker result.

The two runtimes are indistinguishable while cores are free: 0.5084 against 0.5086 at one worker, with both coefficients of variation at or below 0.14%. At one worker per physical core OpenMP is

flat to 0.3%, which reproduces the sixteen-core result of Section 7.4 on a completely different part and a different runtime. At eight workers both runtimes show the hyperthread knee, so the knee belongs to the hardware rather than to either implementation.

The one row where they disagree is four workers, and the deviation belongs to the pool rather than to the generator. On the OpenMP path the team master is itself a worker, so four workers are four runnable threads on four cores. On the pool path the dispatcher is a separate thread that must be woken once per iteration to collect results, so four workers are five threads on four cores. That costs nothing until the workers have claimed every core, which is why the pool measures flat on the sixteen-core host and flat here at one and two workers. This explanation is an inference consistent with the data rather than a separate measurement, and the direct test would need a three-worker row that the benchmark does not register.

A second-order result falls out of the same sweep, and it is recorded because a caller can act on it. OpenMP spins before sleeping at the end of a parallel region. Forcing it to sleep instead costs 9.0% at one worker per physical core, where the spin is free, and saves 4.0% at two workers per core, where every spin steals issue capacity from a sibling that is still working. Callers who run one worker per physical core should leave the default alone; callers who oversubscribe to every hardware thread should measure a passive wait policy.

The practical guidance that follows is short: size a Philox thread pool by physical cores, not by the value `std::thread::hardware_concurrency` returns.

8 Statistical validation

Philox4x32-10 is a published and analysed construction, so the question here is whether this implementation reproduces it, not whether the algorithm is sound. Both were checked.

PractRand to 1 TB. A raw 32-bit stream at seed 0 through the AVX2 backend was tested to 2^{40} bytes, reporting no anomalies in 304 results over 4173 seconds. Exactly one test was flagged, at the 4 GB checkpoint, at PractRand's mildest severity, and it did not recur at any later checkpoint up to 1 TB. The non-recurrence is the substantive evidence rather than the count: PractRand's checkpoints are not independent draws, since each re-runs the same tests over the accumulated stream, so real structure keeps flagging as data accumulates. A flag at 4 GB absent at 256 times the data is a fluctuation.

Backend equivalence. The backends do not produce similar streams, they produce identical ones, and this is a design invariant rather than an observation. A direct byte comparison over the same 2^{40} bytes the battery consumed confirms it for the scalar and AVX2 kernels; being a byte comparison rather than a digest comparison, it needs no collision argument, and digests are recorded alongside for third-party spot checks. For AVX-512 and NEON the same conclusion rests one layer down, on a parity matrix that sweeps carrying counters, edge keys and every block count straddling a plausible SIMD width. A kernel that reproduces the scalar stream bit for bit cannot fail a battery the scalar stream passes.

TestU01 BigCrush. All 160 statistics pass, in 2 hours 31 minutes of processor time. The battery is driven through the engine rather than a byte stream, so the refill buffer and run-time dispatch are under test as well as the kernel.

The float32 stream is deliberately not tested this way. Running the converted float bits through a bit-level battery measures the IEEE-754 format, not the generator. The subtraction in Equation (5) renormalises, so the sign bit is always clear, the exponent field is geometrically skewed with half of all outputs sharing one exponent, and low mantissa bits are frequently zero because renormalising a small result shifts zeros in from the right. vphilox duly fails 114 tests, the lowest bit of each word reporting $p = 2 \times 10^{-1994}$, exactly as the mechanism predicts. Every correct float

generator fails this test; the bits of a uniform float are not uniform bits. The log is archived rather than discarded, so the next person to attempt it finds the explanation.

What the conversion does have to guarantee is tested directly: an exhaustive round trip over all 2^{23} representable outputs, a chi-squared test over 1024 bins bounded five standard deviations in *both* directions, and a Kolmogorov-Smirnov test against the uniform distribution. The two-sided bound on the chi-squared statistic is deliberate, because a value far below the degrees of freedom indicates counts tracking expectation more closely than chance allows, which is what a silently coarsened grid looks like. Mutation testing confirms the tests are not redundant: a stuck low mantissa bit passes chi-squared and Kolmogorov-Smirnov and is caught only by the exact round trip, since halving the grid resolution leaves values uniform at any coarser scale.

8.1 Agreement with an independent implementation

Everything above establishes that vphilox agrees with itself across four kernels and two architectures, and reproduces the published test vectors. Neither addresses the layer at which Philox implementations actually diverge: the round function is fixed by the specification, but the mapping from a user-facing seed onto a counter and a key is not, and that is where two conforming implementations produce different streams from the same arguments.

We therefore cross-check against NVIDIA's cuRAND [7], which seeds with a triple of seed, subsequence and offset where vphilox seeds with a key and a counter. Sixteen cases were compared over 4096 words each, under all three x86 backends, and every case is identical. The cases were chosen to exercise the seeding arithmetic rather than the round function: offsets that are not block aligned, subsequence and offset values straddling 2^{32} so that a carry crosses the counter's low words, the maximum subsequence, and edge keys including all zeros and all ones.

Each case asserts two independent things. The first feeds vphilox the counter and key cuRAND reports for itself after seeding, which tests the round function and the increment order while assuming nothing about the mapping. The second requires the state cuRAND derived to match what the documented mapping predicts from the seed triple, and that is the interoperability claim: the half that could plausibly have failed.

The mapping it confirms is that cuRAND's subsequence occupies the counter's high 64 bits, its offset divided by four the low 64 bits, and the remainder of that division a word index into the resulting block; the asymmetry to watch is that a cuRAND offset counts words where a vphilox counter counts blocks of four. Because cuRAND reserves the whole high half for stream selection, placing its subsequences 2^{66} values apart, a program that partitions across GPU threads the cuRAND way and across CPU threads the vphilox way obtains identical non-overlapping streams.

The comparison needs neither a GPU nor a device compiler: cuRAND guards its function decoration behind a macro, so defining it compiles NVIDIA's reference implementation as host code, and the only target-dependent line computes the high half of a 32-by-32-bit product identically on both paths.

9 Limitations and results that go against the design

xoshiro256++ is faster on four of five machines. It leads on the Pi, on Sapphire Rapids and on both Cascade Lake hosts. On Skylake-SP the vphilox bulk path costs 0.4210 cycles per byte against 0.4703, so vphilox wins one part on one machine, which is a result rather than a trend. We report it and do not optimise toward it: xoshiro256++ is a latency-bound scalar chain a wider kernel does not catch, and it offers none of the properties in Section 1. Speed here has to be good enough that the objection cannot be raised again, no more.

The buffered engine is still behind `std::mt19937` on ARM. The NEON kernel leads it at 1.33×, but the per-call interface costs 1.15× `std::mt19937` per byte because the refill drain now

costs more than the kernel does. Callers who use the bulk path get the full benefit on that platform; callers who use `operator()` do not.

Double-precision conversion is scalar. The bulk float path is vectorized and dispatched exactly as the kernels are, converting a 256-word tile at a time for the gains reported in Section 4.5. The double path is not. Each double consumes two 32-bit words, so a vector form has to combine two input registers for every output register, and that kernel is not written. Callers converting to double therefore pay a per-value cost that callers converting to float no longer do.

The instruction-supply result rests on one four-core host. The measurement in Section 7.5 needs simultaneous access to symmetric multithreading, a working performance monitoring unit and real control over thread placement, and exactly one machine on record provides all three. Its co-located arm exceeds the variation bar, as Section 7.5 states. The elimination of instruction supply is a direction reproduced across four runs and two clock regimes rather than a threshold, and it would be stronger on a host with more cores; the same is true of the runtime contrast in Section 7.6, whose four-worker row is measured where four workers exhaust the machine. Both qualifications from Section 7.5 carry here: ruling instruction supply out is weaker than establishing port contention, and the host gate selects for a large co-location penalty, so 59% describes an admitted host rather than the cost in general.

The cuRAND comparison is against host-compiled reference code. As Section 8.1 states, what is verified is NVIDIA's reference implementation rather than code generated for a device. The residual gap is one line computing the high half of a 32-bit product.

Cloud measurement. Four of the six hosts are cloud instances with dedicated virtual CPUs. Steal time is invisible on such a host, and the mitigation is the variation bar plus the quality label rather than elimination of the risk. The Raspberry Pi 5, one of the two bare-metal hosts, is also where the flattest and most reproducible scaling curve was measured, which is consistent with the concern being real. The other bare-metal host is the four-core laptop just discussed, so the archive holds no machine that is bare metal and large at once.

10 Related work

Salmon et al. [9] introduced the Philox and Threefry families and the argument that a counter-based generator makes parallel random number generation a matter of partitioning integers. The reference implementation, Random123, is the source of the test vectors used here. This work differs in what it targets: portable serialized state as the deliverable, with vectorization as the means of making that state free rather than as the result.

The Mersenne Twister [5] remains the default engine in the C++ standard library, and its state is the object of the portability failure in Section 1. PCG [8] and xoshiro/xoroshiro [1] are modern stateful generators with small states and strong statistical results; both appear in the comparison matrix, and both are faster than vphilox on most parts measured. Neither offers a standard-library-independent serialized state, constant-time seek or thread-count independence, which is the trade this paper is about.

TestU01 [4] and PractRand [3] supply the batteries in Section 8. Vendor libraries including Intel's Vector Statistics Library and NVIDIA's cuRAND [7] ship vectorized Philox variants; the distinction here is the header-only, dependency-free distribution model and the run-time dispatch that lets one binary select its kernel on the host it lands on.

11 Conclusion

Checkpointing a `std::mt19937` and restoring it on a different standard library is not reliable, and the failure mode on macOS is silent. Storing a position instead of an implementation removes the failure by construction, and the position of a counter-based stream is a key and an integer.

What has kept that substitution off CPUs is a performance objection, and the measurements here answer it in two parts. Its stated magnitude, a tenfold penalty against `std::mt19937`, did not reproduce on any of five processors; the measured range is 1.10× to 1.65×. Interleaving counters across SIMD lanes then brings the bulk path to between 0.22× and 0.75×, and avoids the AVX-512 frequency licence trap by construction, since Philox contains no floating-point instruction. In parallel, cost per byte does not move with thread count, and the first limit on a sixteen-core host is hyperthread co-location. Measurement narrows what that limit can be without pinning it down: instruction-cache misses fall by 36% where the penalty appears, so instruction supply does not fit, and a second threading runtime reproduces the flat result to within 0.3%, so the knee is not an artefact of this project's worker pool. Execution-port contention is what remains, an inference from the instruction mix rather than a measurement of its own.

Two directions follow. The refill buffer is now the dominant remaining cost on the fastest kernels, at roughly 109% of bulk throughput on AVX-512, so removing the second pass over every byte is worth more than widening the kernel further. And the double-precision conversion should move onto the vector path: the single-precision path is already there and dispatched as the kernels are, while doubles are not, each consuming two 32-bit words so that a vector form must combine two input registers per output register.

Availability

vphilox is released under MIT or Apache-2.0. Every benchmark in this paper is reproducible from the archived JSON and the scripts in the repository, and the derived tables and figures are regenerated by a single command that continuous integration verifies. The release measured here is archived under a DOI. The repository URL and that DOI both name the author, so both are withheld here and are on the title page supplied to the editor, who can pass them to reviewers.

References

[1] David Blackman and Sebastiano Vigna. 2021. Scrambled Linear Pseudorandom Number Generators. *ACM Trans. Math. Software* 47, 4 (2021), 36:1–36:32. doi:10.1145/3460772

[2] CharlesHMJr. 2026. XGBoostError: Input Stream Corrupted When Unpickling a Linux-Trained Booster on Windows (xgboost 3.3.0). dmlc/xgboost issue #12459, GitHub. https://github.com/dmlc/xgboost/issues/12459 Accessed 26 August 2026.

[3] Chris Doty-Humphrey. [n. d.]. PractRand: Practically Random, C++ Library of Statistical Tests for RNGs. Release PractRand-pre0.95, which reports its own version as 0.95. https://sourceforge.net/projects/pracrand/ Accessed 26 August 2026.

[4] Pierre L'Ecuyer and Richard Simard. 2007. TestU01: A C Library for Empirical Testing of Random Number Generators. *ACM Trans. Math. Software* 33, 4 (2007), 22:1–22:40. doi:10.1145/1268776.1268777

[5] Makoto Matsumoto and Takuji Nishimura. 1998. Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudo-Random Number Generator. *ACM Transactions on Modeling and Computer Simulation* 8, 1 (1998), 3–30. doi:10.1145/272991.272995

[6] Rory Mitchell. 2026. Review Comment on Pull Request #12485, "Make the Serialized RNG State Portable Across Platforms". dmlc/xgboost, GitHub. https://github.com/dmlc/xgboost/pull/12485#issuecomment-5355357794 Accessed 26 August 2026.

[7] NVIDIA Corporation. 2026. *cuRAND Library Programming Guide.* https://docs.nvidia.com/cuda/curand/ cuRAND 10.4.1, CUDA Toolkit 13.1.

[8] Melissa E. O'Neill. 2014. *PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation.* Technical Report HMC-CS-2014-0905. Harvey Mudd College, Claremont, CA, USA.

[9] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. 2011. Parallel Random Numbers: As Easy as 1, 2, 3. In *Proceedings of the 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11).* ACM, New York, NY, USA, 16:1–16:12. doi:10.1145/2063384.2063405

Received 9 October 2026